SIGPwny

Embedded FA2026 • 2026-11-09

Bootloader/Firmware Security

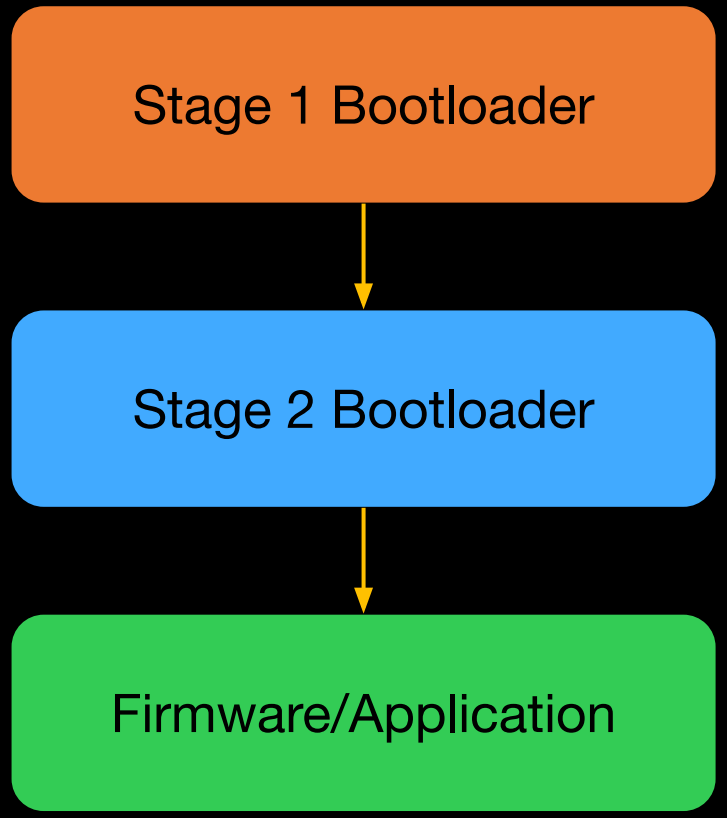
Swetha and Krishnan

Agenda

- Basics
  - Review
  - Why does firmware security matter?
  - Secure bootloaders
- Attacks
- Extracting code
- Case studies
  - STM32F217 DFU Exit
  - MAX78000 Firmware Extraction



Bootloader Review



Hardcoded in the microcontroller by manufacturer

Provided by firmware distributor for additional features/security

The actual program from the distributor



Why do bootloaders need to be secure?

- Bootloaders have the highest level of privilege in embedded systems
- If the bootloader is compromised, other levels of privilege are essentially meaningless
- Direct control of peripherals and hardware
- Protecting intellectual property, credentials/secrets, etc.
- Operational technology (OT) means real-world impacts
- Stealth, persistence



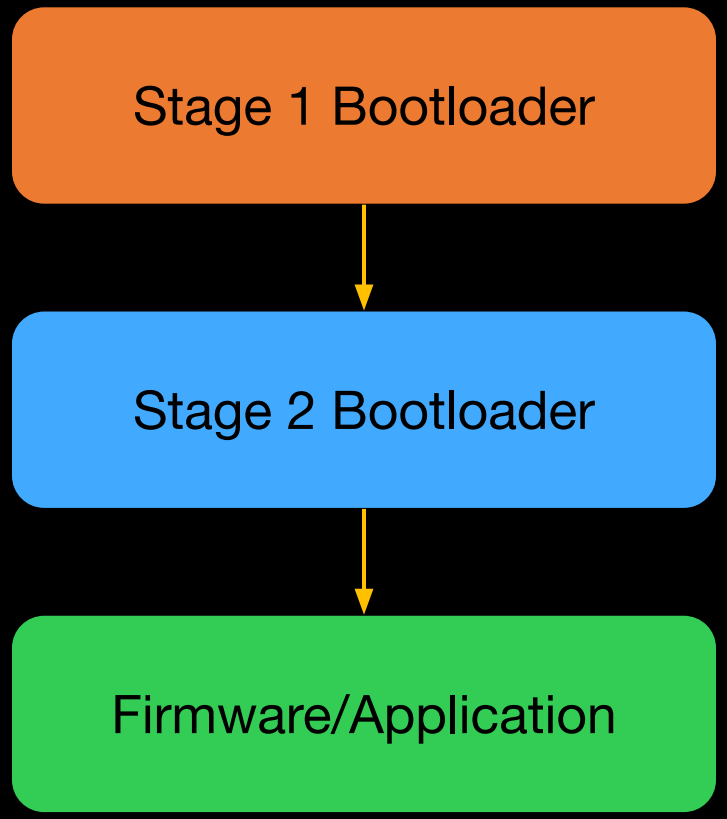
Why does firmware need to be secure?

- Some control of peripherals and hardware
- Protecting intellectual property, credentials/secrets, etc.
- Operational technology (OT) means real-world impacts



Bootloader Responsibilities

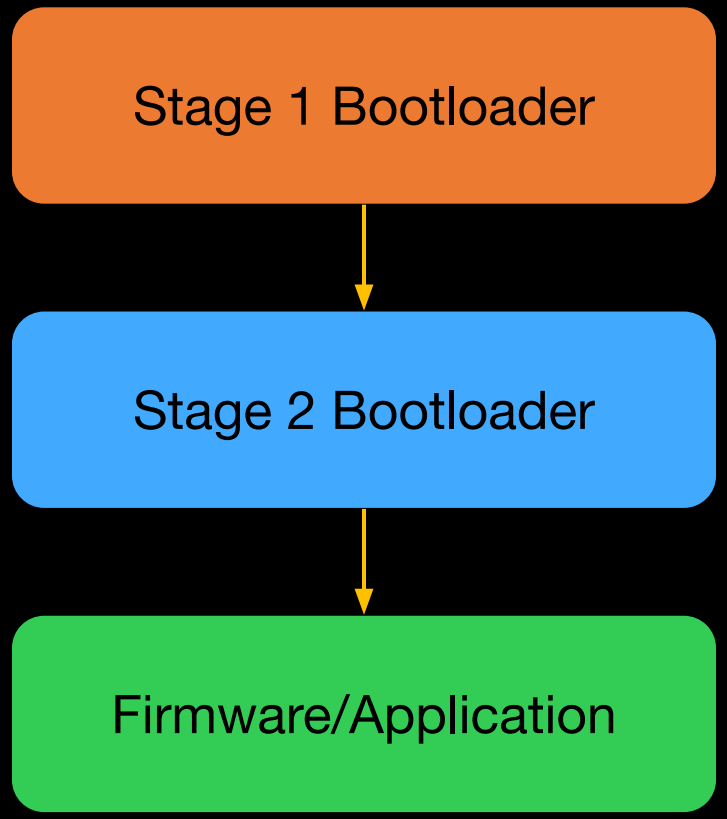
- Flashing code





Bootloader Responsibilities

- Flashing code
- Device recovery/device firmware update (DFU)





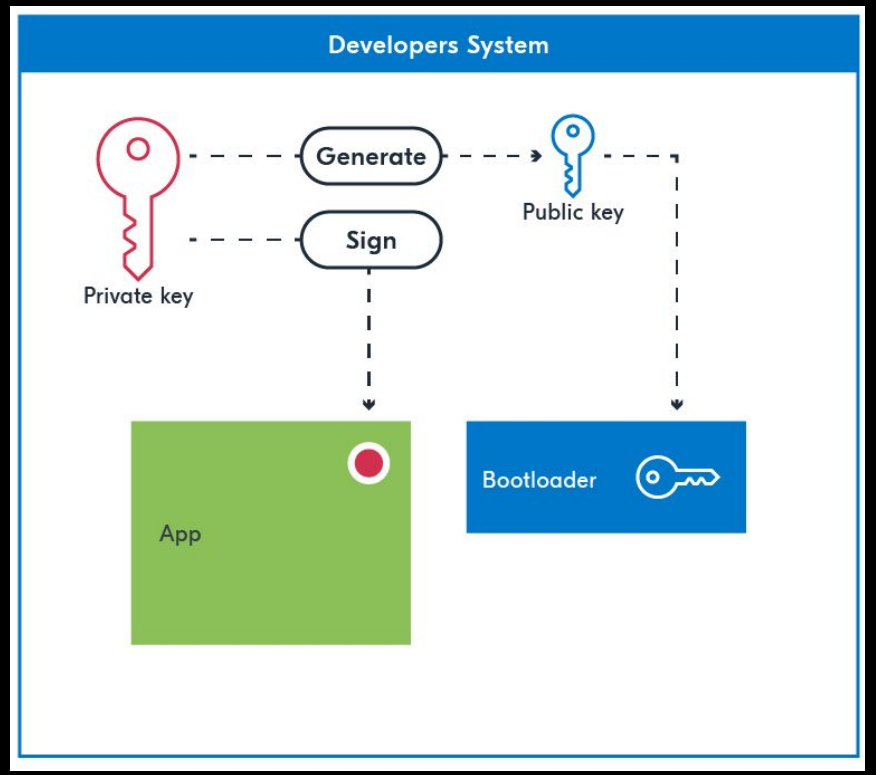
Secure Bootloader Responsibilities

- Flashing code
- Device recovery/device firmware update (DFU)
- Verifying flashed code



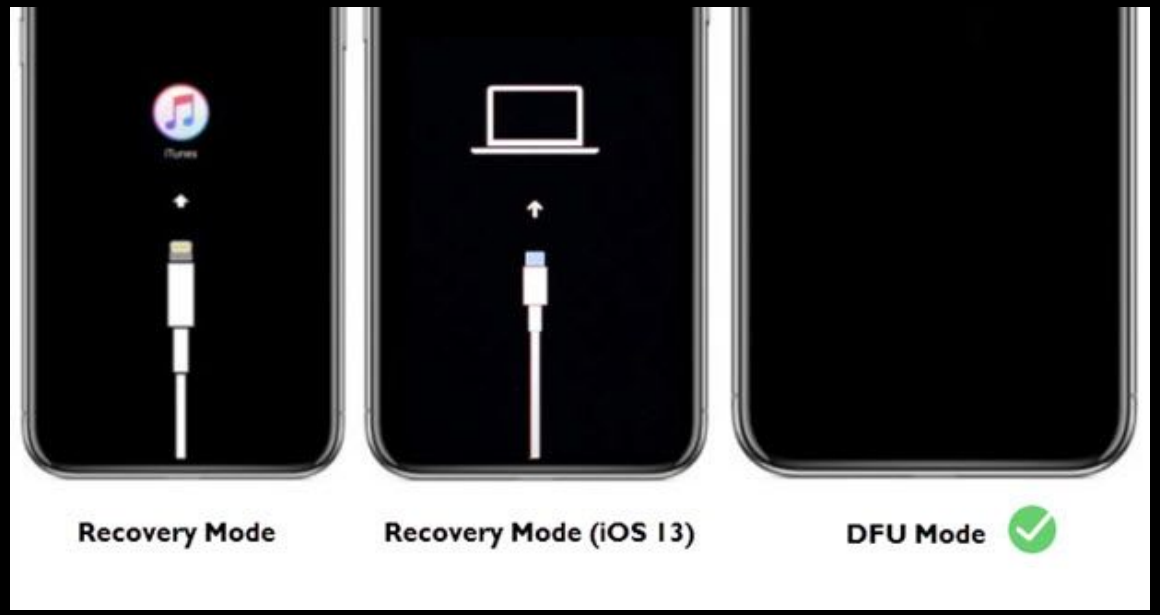
Secure Bootloader Responsibilities

- Flashing code
- Device recovery/device firmware update (DFU)
- Verifying flashed code
- Only allowing signed code to be flashed
- Executing the code once requirements are met



Secure Bootloader Responsibilities

- Flashing code
- Device recovery/device firmware update (DFU)
- Verifying flashed code
- Only allowing signed code to be flashed
- Executing the code once requirements are met





HW that Supports Secure Bootloaders

- TPM (Trusted Platform Module) - stores keys for encryption in hardware, serving as an immutable hardware root of trust
  - Infineon SLB 9670
  - Nuvoton NPCT750
  - STM ST33 Series
  - Used by Windows BitLocker

- Secure Enclaves & Trusted Execution Environments - secure area of processor meant for sensitive code
  - Apple Secure Enclave
  - Intel SGX
  - Arm TrustZone (only available on Cortex M33+ and other chips)



Attacks



Generally, 4 levels of attacks

- Misconfigurations
  - The firmware does actually provide some security settings, but the distributor of the MCU did not lock it down
- Logic bugs
  - The firmware provides you more access than should have even in locked down settings, allowing you to do bad things
- Implementations bugs
  - E.g. buffer overflows, traditional exploits
  - Generally requires access to firmware binary to reverse engineer/audit
- Hardware attacks
  - Fault injection/voltage glitching, side-channel analysis
  - Also usually requires access to firmware binary



Do we actually need firmware code?

- For the bootloader, just read the MCU user manual to understand how it works!

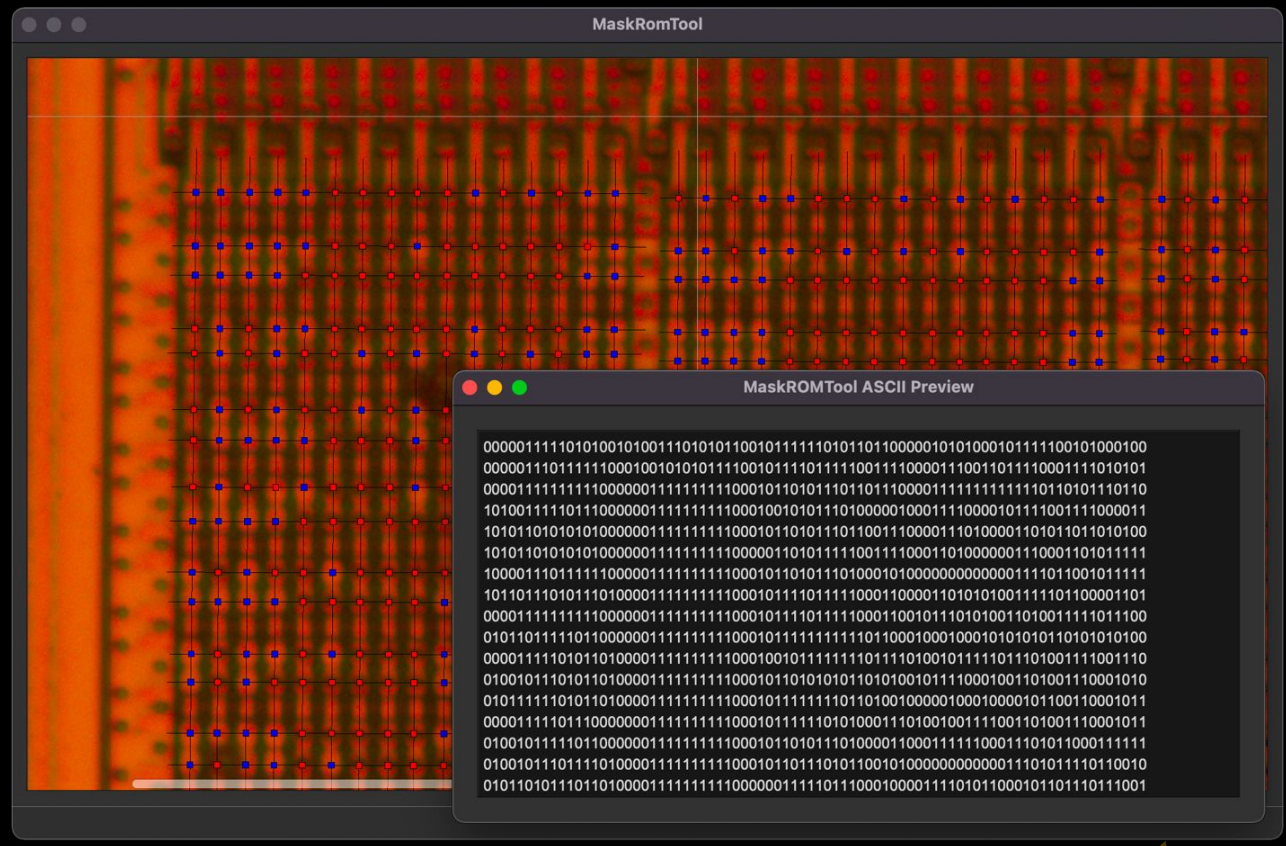
- If you can't find any misconfigurations or logic vulnerabilities, then maybe look into getting code so you can:
  1. Find implementation bugs (e.g. buffer overflows)
  2. Discover hidden functionality (manufacturers love this)
  3. Have assembly to perform fault injection



Getting bootloader/firmware code

Some options:

1. On your own chip, enter debug mode and read out the region of memory which corresponds to bootloader
2. Find an exploit in target MCU and run code to dump bootloader and/or firmware
3. Decap the chip and read out the ROM mask!





Getting bootloader/firmware code

Some options:

1. On your own chip, enter debug mode and read out the region of memory which corresponds to bootloader
2. Find an exploit in target MCU and run code to dump bootloader and/or firmware
3. Decap the chip and read out the ROM mask!

Pros/Cons:

\+ Very reliable
\- Can only get bootloader code
\- Code may not be same as target
\- Bootloader region may have read off

\+ Requires exploit
\- Relevant region may have read off

\- Destructive
\- Requires lab chemicals
\- Only works for ROM mask (Stage 1 Bootloader)



Case Studies



Generally, 4 levels of attacks

- Misconfigurations
  - The bootloader does actually provide some security settings, but the distributor of the MCU did not lock it down
- Logic bugs
  - The bootloader provides you more access than should have even in locked down settings, allowing you to do bad things
- Implementations bugs
  - E.g. buffer overflows, traditional exploits
  - Generally requires access to firmware binary to reverse engineer/audit
- Hardware attacks
  - Fault injection/voltage glitching, side-channel analysis
  - Also usually requires access to firmware binary



Case Studies



Case Study: STM32F217 DFU Exit

- USB DFU bootloader properties:
  - JTAG allows us to write applications into unused SRAM
  - We can set the bootloader's address pointer

What's the issue with this?



Case Study: STM32F217 DFU Exit

1. Place shellcode in SRAM, persists even after reset

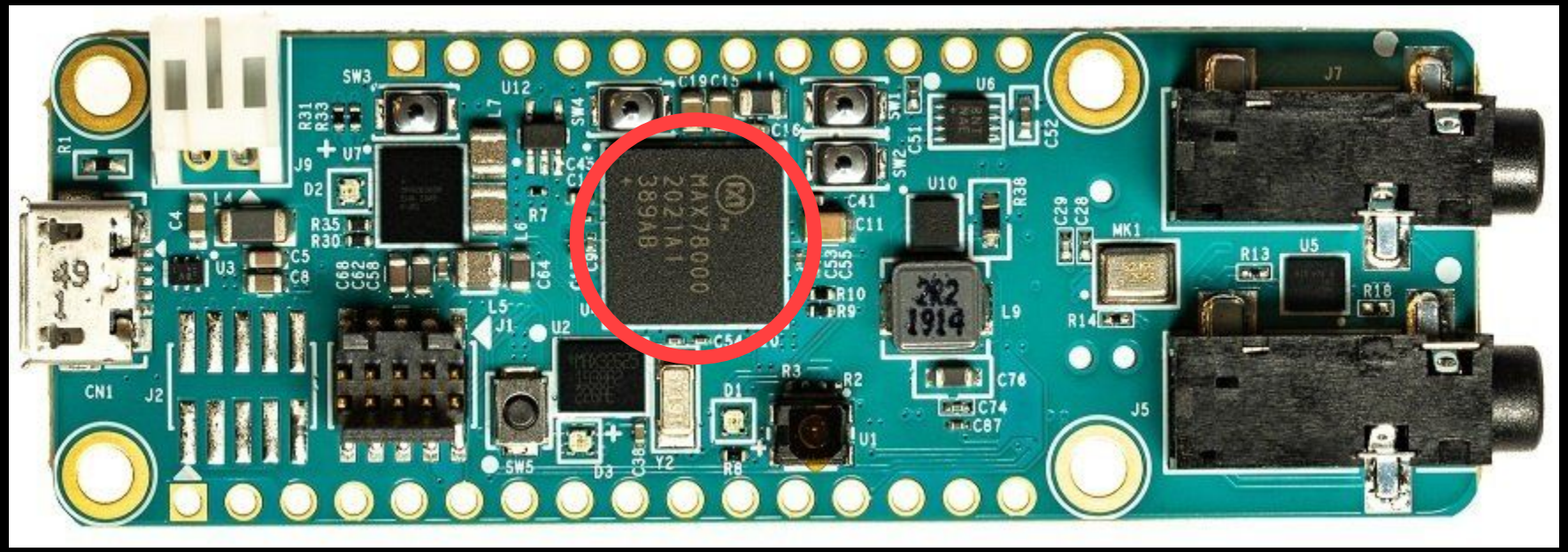
2. Set address pointer (address of our shellcode in SRAM) of bootloader

3. Bootloader jumps to our shellcode and begins executing



Case Study: MAX78000

- The MAX78000 was used in eCTF 2024 and eCTF 2025



26. Bootloader

The ROM bootloader provides for program loading and verification. The physical interface between the external host and the bootloader defaults to the UART.

The secure bootloader (SBL) employs a hash-based message authentication code (HMAC SHA-256) to guarantee both the authenticity and downloaded program files and the integrity of internal program memory after each reset.

All versions of the bootloader provide the ability to block read/write access to program memory.

Bootloader features:

- Common functionality of the bootloader and secure bootloader
- Checksum verification of the ROM image before further ROM execution.
- SWD is disabled in LOCKED and PERMLOCKED states.
- Programmable through the UART or SWD interface.
- UART operates at 115200 bps.
- LOCKED mode disables the SWD and disallows any change to flash through the bootloader.
- Unlock erases all flash and secrets before unlocking SWD.
- Optional PERMLOCKED state only allows for program validation lock.

Secure Bootloader (SBL) features:

- Secure HMAC SHA-256 using secret key-based transactions.
- The trusted secure boot provides automatic program memory verification and authentication before execution after every reset.
- Integrity and authentication verification of program memory downloads
- Optional challenge/response gating entry to the bootloader

MAX78000 User Guide



26.2 Bootloader Operating States

Each bootloader supports the modes shown in *Table 26-2*. Each bootloader mode has a unique prompt.

Table 26-2: The Bootloader Operating States and Prompts

State	Bootloader	Secure Bootloader	Recognized Commands	Prompt
UNLOCKED	Yes	Yes	All Commands U/L/P	"ULDR> " <0x55> <0x4C> <0x44> <0x52> <0x3E> <0x20>
LOCKED	Yes	Yes	Only L/P	"LLDR> " <0x4C> <0x4C> <0x44> <0x52> <0x3E> <0x20>
PERMLOCKED	Yes	Yes	Only P	"PLLDR> " <0x50> <0x4C> <0x4C> <0x44> <0x52> <0x3E> <0x20>
CHALLENGE	No	Yes	*GC – Get Challenge* *SR – Send Response*	"CR> " <0x43> <0x52> <0x3E> <0x20>
APPVERIFY	No	Yes	N/A	N/A

The *LOCK – Lock Device* and *PLOCK – Permanent Lock* commands do not change the bootloader prompt or take effect until the bootloader is reset.



26.2.1 UNLOCKED

The UNLOCKED state provides access to load secure keys and configuration information. Program loading, verification, and status are available in the UNLOCKED state. The SWD interface is available for use.

Transitioning from the LOCKED to UNLOCKED states erases all program memory. In the SBL, it also clears the challenge/response and application keys stored by the SBL.

The challenge and application keys can be erased by executing the Unlock command while resetting the device in the UNLOCKED state. This eliminates the need to transition through the LOCKED state.

26.2.2 LOCKED

The LOCKED state disables access to program memory other than to verify it. The application and challenge/response keys cannot be changed without first changing to the UNLOCKED state.

For the SBL, if the optional challenge key is activated, the bootloader starts in the CHALLENGE state. Successfully completing the challenge/response transitions to the previous PERMLOCKED or LOCKED state.

The application key should be configured before executing the *LOCK – Lock Device* command.

26.2.3 PERMLOCKED

The PERMLOCKED state disables access to program memory other than to verify it with a simple SHA-256 hash. The commands available in the PERMLOCKED state are shown in *Table 26-3*.

Table 26-3: PERMLOCK Command Summary

Command
H – Check Device
I – Get ID

For the SBL, if the optional challenge key is activated, the bootloader starts in the CHALLENGE state. Successfully completing the challenge/response transitions to the previous PERMLOCKED state.

The application key should be configured before executing the *PLOCK – Permanent Lock* command.



26.8 General Commands

Table 26-5: MAX78000 General Command Summary

Command
L - Load
P – Page Erase
V – Verify
LOCK – Lock Device
PLOCK – Permanent Lock
UNLOCK – Unlock Device
H – Check Device
I – Get ID
S – Status
Q – Quit

Suppose the device is in permanent lock (PLOCK) state...
What commands can we run?



Table 26-12: H – Check Device

H – Check Device	Perform SHA-256 Hash Over Memory Range
Description	Performs a simple SHA-256 (not HMAC SHA-256) hash of bytes starting at 32-bit address 0xnnnnnnnn to 0xmmmmmmmm. The minimum hash input size is 512 bytes. The function returns a 32-byte hash value.
Modes	U/L/P
Command	H 0xnnnnnnnn 0xmmmmmmmm<0x0D><0x0A>
Response: Success	yyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyyy<0x0D><0x0A>
Response: Failure	<0x0D><0x0A>



MAX78000 Firmware Extraction



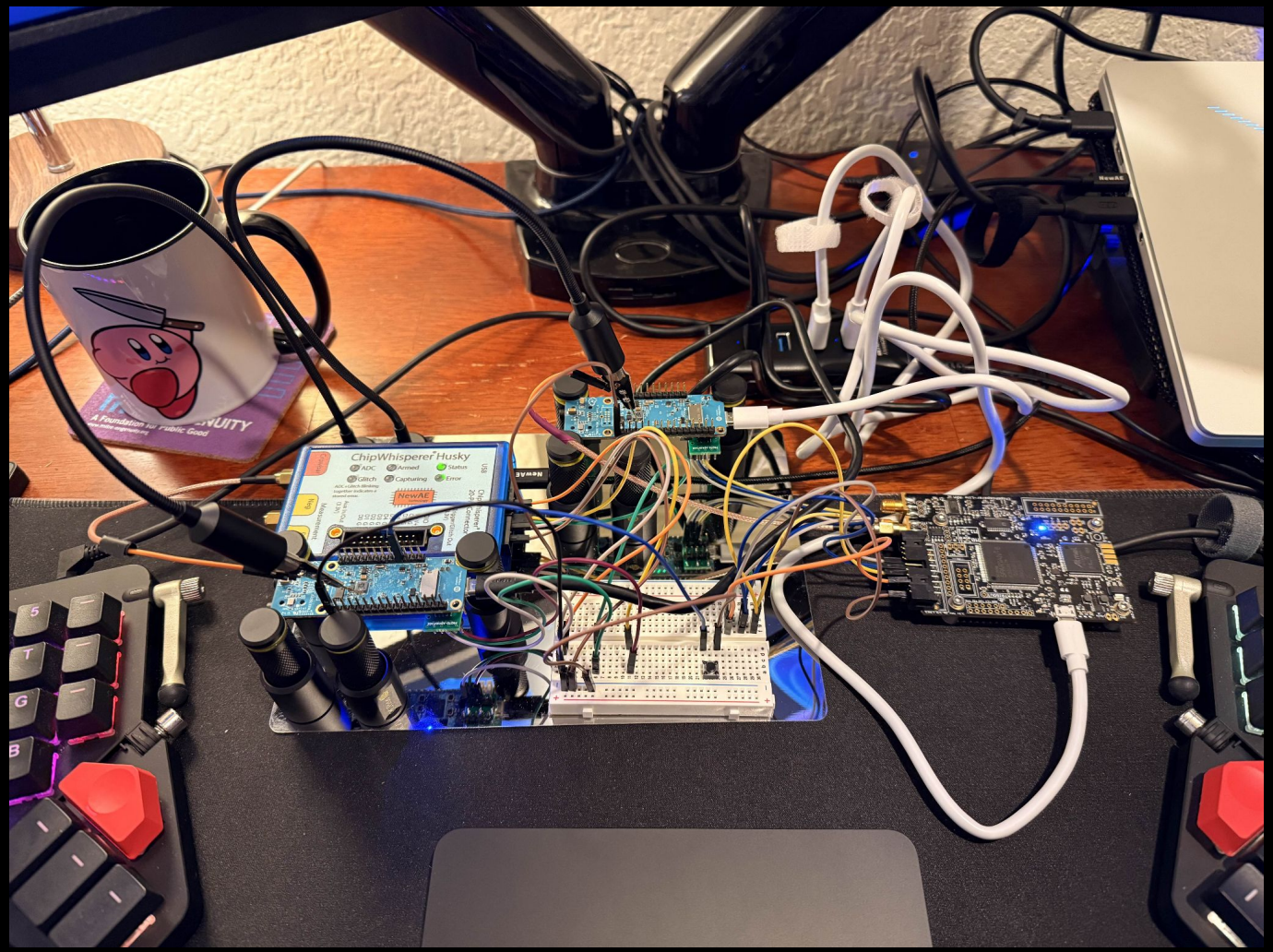
```
void main_rom_bootloader(void) {
  int ret;
  ret = is_locked();
  if (ret == 0) goto BL_MODE_UNLOCKED;
  do {
    disable_debug();
    ret = is_chal_response_locked();
    if (ret != 0) goto BL_MODE_CHALLENGE_RESPONSE;
    while (true) {
      ret = is_permlocked();
      if (ret != 0) mode_permlocked();
      mode_locked();
BL_MODE_UNLOCKED:
      ret = is_permlocked();
      if (ret != 0) break;
      enable_debug();
      mode_unlocked();
BL_MODE_CHALLENGE_RESPONSE:
      mode_challenge_response();
    }
  } while (true);
}
```

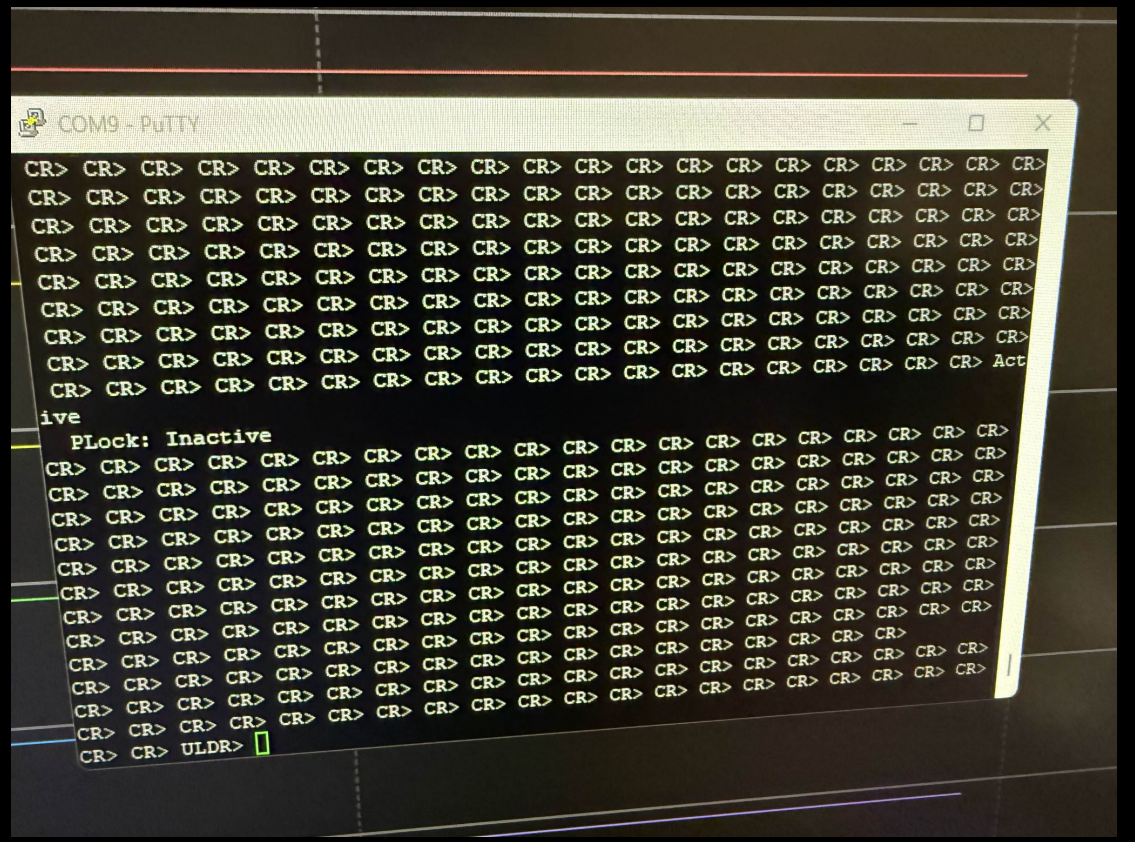


```
void main_rom_bootloader(void) {
  int ret;
  ret = is_locked();
  if (ret == 0) goto BL_MODE_UNLOCKED;
  do {
    disable_debug();
    ret = is_chal_response_locked();
    if (ret != 0) goto BL_MODE_CHALLENGE_RESPONSE;
    while (true) {
      ret = is_permlocked();
      if (ret != 0) mode_permlocked();
      mode_locked();
BL_MODE_UNLOCKED:
      ret = is_permlocked();
      if (ret != 0) break;
      enable_debug();
      mode_unlocked();
BL_MODE_CHALLENGE_RESPONSE:
      mode_challenge_response();
    }
  } while (true);
}
```

Use glitching to bypass CR mode and get into any of these states!




ChipWhisperer Husky
NewAE


COM9 - PuTTY
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> Act
ive
PLock: Inactive
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR> CR>
CR> CR> ULDR>

Next Meetings

2026-11-16 • **Next Monday**

- Side Channel Attacks + Lab



Meeting content can be found at sigpwny.com/meetings.



SIGPwny